

CS432: Databases

Structured Query Language (SQL)

Instructor
Yogesh K. Meena

Lecture no.
6

CSE, IIT Gandhinagar
January 28, 2026

Structured Query Language (SQL)

Not just a query language, it also provides a way to:

- define the structure of the data,
- modify data in the database,
- specify security constraints.



A brief history

IBM developed the original version of SQL (Sequel) around 1970s.

- Later the name was evolved to SQL.
- American National Standards Institute (ANSI) and the International Organization for Standardization (ISO) published an SQL standard SQL-86 in 1986.
- SQL-89, SQL-92, SQL:1999, SQL:2003, SQL:2006, SQL:2008, SQL:2011, SQL:2016, and SQL:2019.
- Commercial systems offer most, if not all, SQL-92 features, plus varying feature sets from later standards and special proprietary features.
- **Not all examples here may work on your particular system.**

Components of SQL

- **Data-definition language (DDL)**
 - defining relation schemas
 - deleting relations
 - modifying relation schemas
- **Data-manipulation language (DML)**
 - query database
 - insert tuples
 - delete tuples
 - modify tuples

Components of SQL

- **Integrity** – SQL DDL to specify integrity constraints
- **View definition** – SQL DDL to define views
- **Transaction control** – to specify the beginning and ending of transactions.
- **Embedded SQL and dynamic SQL** – how SQL statements can be embedded within C, C++, and Java. *[Python and modern languages like Node.js or Go have shifted almost entirely toward Dynamic SQL and ORMs (Object-Relational Mappings)]
- **Authorization** – SQL DDL to specify access rights to relations and views.

Embedded SQL and dynamic SQL

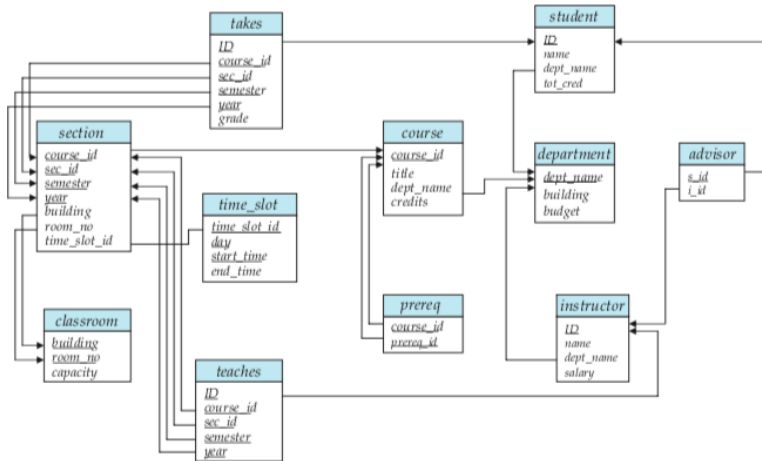
Example: Modern Language Integration with Databases

Language	Primary Method	Tool/Library
Python	Dynamic SQL / ORM	SQLAlchemy, psycopg2, Django ORM
C#	LINQ / ADO.NET	Entity Framework
Node.js	Dynamic SQL (Async)	Sequelize, Prisma, node-postgres
Go	Dynamic SQL / Struct Tags	database/sql, GORM

Domain Types in SQL

- **char(n)**: Fixed length character string, with user-specified length n.
- **varchar(n)**: Variable length character strings, with user-specified maximum length n.
- **int**: Integer (a finite subset of the integers that is machine-dependent).
- **smallint**: Small integer (a machine-dependent subset of the integer domain type).
- **numeric(p,d)**: Fixed point number, with user-specified precision of p digits, with d digits to the right of decimal point. (ex., numeric(3,1), allows 44.5 to be stores exactly, but not 444.5 or 0.32)
- **real, double precision**: Floating point and double-precision floating point numbers, with machine-dependent precision.
- **float(n)**: Floating point number, with user-specified precision of at least n digits.
- **null**: an absent value

Recall: Schema Diagrams for University Database



Create Table Construct

- An SQL relation is defined using the create table command:

```
create table r( $A_1D_1$ ,  $A_2D_2$ , ...,  $A_nD_n$ ,  
    (integrity-constraint1),  
    ...,  
    (integrity-constraintk))
```

- r is the name of the relation
 - each A_i is an attribute name in the schema of relation r
 - D_i is the data type of values in the domain of attribute A_i
- Example:

```
create table instructor (  
    ID          char(5),  
    name        varchar(20),  
    dept_name   varchar(20),  
    salary      numeric(8,2));
```

Integrity Constraints in Create Table

- not null – Null value is not allowed for that attribute
- primary key ($A_1, \dots, A_n$) – nonnull and unique
- foreign key ($A_m, \dots, A_n$) references $r - (A_m, \dots, A_n)$ for any tuple in the relation must correspond to values of the primary key attributes of some tuple in relation r .
- Example:

```
create table instructor (  
    ID          char(5),  
    name        varchar(20) not null,  
    dept_name   varchar(20),  
    salary      numeric(8,2),  
    primary key (ID),  
    foreign key (dept_name) references department);
```

primary key declaration on an attribute automatically ensures **not null**

More Relation Definitions for University Database

example:

```
create table student (  
    ID          varchar(5),  
    name       varchar(20) not null,  
    dept_name  varchar(20),  
    tot_cred   numeric(3,0),  
    primary key (ID),  
    foreign key (dept_name) references department);
```

More Relation Definitions for University Database

example:

```
create table takes (  
    ID          varchar(5),  
    course_id   varchar(8),  
    sec_id      varchar(8),  
    semester    varchar(6),  
    year        numeric(4,0),  
    grade       varchar(2),  
    primary key (ID, course_id, sec_id, semester, year) ,  
    foreign key (ID) references student,  
    foreign key (course_id, sec_id, semester, year) references section);
```

Note: *sec_id* can be dropped from primary key above, to ensure a student cannot be registered for two sections of the same course in the same semester

More Relation Definitions for University Database

example:

```
create table course (  
    course_id      varchar(8),  
    title          varchar(50),  
    dept_name      varchar(20),  
    credits        numeric(2,0),  
    primary key (course_id),  
    foreign key (dept_name) references department);
```

Updates to tables

- **Populating table**
 - **insert into** instructor **values** ('10211', 'Smith', 'Biology', 66000);
 - The values are specified in the order in which the corresponding attributes are listed in the relation schema.
- **Delete tuple**
 - Remove all tuples from the student relation
 - **delete** from student;
- **Delete Table**
 - Deletes tuples as well the schema of the student relation.
 - **drop** table student;
- **Update table**
 - **alter** table r **add** A D;
 - where A is the name of the attribute to be added to relation r and D is the domain of A.
 - All exiting tuples in the relation are assigned null as the value for the new attribute.
 - **alter** table r **drop** A;
 - where A is the name of an attribute of relation r
 - Dropping of attributes not supported by many databases.

Basic Query Structure

- A typical SQL query has the form:
select $A_1, A_2, \dots, A_n$
from $r_1, r_2, \dots, r_m$
where P
 - A_i represents an attribute
 - R_i represents a relation
 - P is a predicate.
- The result of an SQL query is a relation.

The select Clause

- The select clause lists the attributes desired in the result of a query:
E.g., find the names of all instructors:

```
select name  
from instructor
```

- **NOTE:** SQL names are case insensitive (i.e., you may use upper- or lower-case letters.). E.g., Name $\equiv$ NAME $\equiv$ name
 - results of '**select**' name **from** instructor?
 - results of '**select**' dept_name **from** instructor?

SQL allows duplicates in relations as well as in query results. To force the elimination of duplicates, insert the keyword **distinct** after select.

– e.g., Find the department names of all instructors, and remove duplicates?

The select Clause

- The select clause lists the attributes desired in the result of a query:
E.g., find the names of all instructors:

```
select name  
from instructor
```

- **NOTE:** SQL names are case insensitive (i.e., you may use upper- or lower-case letters.). E.g., Name $\equiv$ NAME $\equiv$ name
 - results of '**select**' name **from** instructor?
 - results of '**select**' dept_name **from** instructor?

SQL allows duplicates in relations as well as in query results. To force the elimination of duplicates, insert the keyword **distinct** after select.

– e.g., Find the department names of all instructors, and remove duplicates?

```
select distinct dept_name  
from instructor
```

The select Clause (Cont.)

- The keyword all specifies that duplicates should not be removed.

```
select all dept_name  
from instructor
```

- An asterisk in the select clause denotes 'all attributes'

```
select*  
from instructor
```

- An attribute can be a literal with no from clause

```
select* '437'
```

- An attribute can be a literal with no from clause

```
select '437'
```

- An attribute can be a literal with from clause

```
select 'A'  
from instructor
```

The select Clause (Cont.)

- The select clause can contain arithmetic expressions involving the operation, $+$, $-$, $*$, $/$, and operating on constants or attributes of tuples.

```
select ID, name, salary/12  
from instructor
```

- Would return a relation that is the same as the instructor relation, except that the value of the attribute salary is divided by 12.
- Can rename 'salary/12' using the **as** clause:

```
select ID, name, salary/12 as monthly_salary
```

The where Clause

- The where clause specifies conditions that the result must satisfy
E.g., To find all instructors in Comp. Sci. dept
select name
from instructor
where dept_name = 'Comp. Sci.'
- Comparison results can be combined using the logical connectives and, or, and not. E.g., To find all instructors in Comp. Sci. dept with salary > 80000

The where Clause

- The where clause specifies conditions that the result must satisfy
E.g., To find all instructors in Comp. Sci. dept
select name
from instructor
where dept_name = 'Comp. Sci.'
- Comparison results can be combined using the logical connectives and, or, and not. E.g., To find all instructors in Comp. Sci. dept with salary > 80000
select name
from instructor
where dept_name = 'Comp. Sci.' and salary > 80000
- Comparisons can be applied to results of arithmetic expressions.

The from Clause

- The from clause lists the relations involved in the query
E.g., Find the Cartesian product instructor X teaches
select*
from instructor, teaches
- generates every possible instructor – teaches pair, with all attributes from both relations.
- For common attributes (e.g., ID), the attributes in the resulting table are renamed using the relation name (e.g., instructor.ID)
- Cartesian product not very useful directly, but useful combined with where-clause condition (selection operation in relational algebra).

Acknowledgments/Contributions

- Some of the images utilized in these slides are subject to copyright - Abraham Silberschatz, Henry Korth, and S. Sudarshan. Database System Concepts. 6th Edition, McGraw-Hill Education
- Contributor 2024-25, 2025-26 - Yogesh K. Meena, IIT Gandhinagar